Министерство образования Республики Беларусь

Учреждение образования

«Брестский государственный технический университет»

Кафедра интеллектуальных информационных технологий

Лабораторная работа №3

По дисциплине «Интеллектуальный анализ данных»

Тема: «Предобучение нейронных сетей с использованием автоэнкодерного подхода»

Выполнил:

Студент 4 курса

Факультета ЭИС

Группы ИИ-26

Данилюк Т.В.

Проверил:

Никитин С.К.

Брест 2026

Цель работы: научиться осуществлять предобучение нейронных сетей с помощью автоэнкодерного подхода

Общее задание

1. Взять за основу любую сверточную или полносвязную архитектуру с количеством слоев более 3. Осуществить ее обучение (без предобучения) в соответствии с вариантом задания. Получить оценку эффективности модели, используя метрики, специфичные для решаемой задачи (например, MAPE – для регрессионной задачи или F1/Confusion matrix для классификационной).

2. Выполнить обучение с предобучением, используя автоэнкодерный подход, алгоритм которого изложен в лекции. Условие останова (например, по количеству эпох) при обучении отдельных слоев с использованием автоэнкодера выбрать самостоятельно.

3. Сравнить результаты, полученные при обучении с/без предобучения, сделать выводы.

4. Выполните пункты 1-3 для датасетов из ЛР 2.

5. Оформить отчет по выполненной работе, загрузить исходный код и отчет в соответствующий репозиторий на github.

Задание по вариантам

№	Выборка	Тип задачи	Целевая переменная
17	https://archive.ics.uci.edu/dataset/189/parkinsons+telemonitoring	регрессия	motor_UPDRS

Ход работы

Код программы:

```
import copy
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import urllib.request
import io
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, TensorDataset
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

# --- Параметры ---
HIDDEN = [64, 32, 16]
AE_EPOCHS = 30
FT_EPOCHS = 60
BATCH = 64
LR_AE = 1e-3
LR_FT = 1e-3
SEEDS = [0, 1, 2, 3, 4]

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Используемое устройство: {device}")


# --- 1. Загрузка данных Parkinsons Telemonitoring ---
def load_parkinsons():
    print("Загрузка данных Parkinsons Telemonitoring...")
    url = "https://archive.ics.uci.edu/ml/machine-learning-
databases/parkinsons/telemonitoring/parkinsons_updrs.data"
    req = urllib.request.urlopen(url)
    data = req.read().decode('utf-8')

    df = pd.read_csv(io.StringIO(data))

    # Целевая переменная
    y = df["motor_UPDRS"].values.astype(np.float32)

    # Удаляем целевую переменную и другие ненужные для регрессии признаки (subject#, age, sex)
    # Оставляем только клинические и голосовые признаки
    X = df.drop(columns=["subject#", "motor_UPDRS", "total_UPDRS", "test_time", "age",
"sex"]).values.astype(np.float32)
```

```
    return X, y


# --- 2. Подготовка данных ---
def prepare(X, y, seed):
    scaler = StandardScaler()
    X_scaled = scaler.fit_transform(X)

    X_tr, X_te, y_tr, y_te = train_test_split(X_scaled, y, test_size=0.2, random_state=seed)

    X_tr_t = torch.tensor(X_tr, dtype=torch.float32)
    y_tr_t = torch.tensor(y_tr, dtype=torch.float32).view(-1, 1)
    X_te_t = torch.tensor(X_te, dtype=torch.float32)
    y_te_t = torch.tensor(y_te, dtype=torch.float32).view(-1, 1)

    return X_tr_t, y_tr_t, X_te_t, y_te_t


# --- 3. Архитектура сети (4 слоя) ---
class MLP(nn.Module):
    def __init__(self, input_dim, hidden_dims):
        super().__init__()
        self.layers = nn.ModuleList()

        # Входной слой
        self.layers.append(nn.Linear(input_dim, hidden_dims[0]))
        # Скрытые слои
        for i in range(len(hidden_dims) - 1):
            self.layers.append(nn.Linear(hidden_dims[i], hidden_dims[i + 1]))
        # Выходной слой (1 нейрон для регрессии)
        self.out = nn.Linear(hidden_dims[-1], 1)

        self.relu = nn.ReLU()

    def forward(self, x):
        for layer in self.layers:
            x = self.relu(layer(x))
        return self.out(x)


# --- 4. Предобучение автоэнкодером (слой за слоем) ---
def pretrain_layers(model, X_train, seed):
    torch.manual_seed(seed)
    ae_losses = []
    H = X_train.clone()

    # Проходим по всем скрытым слоям
    for i, layer in enumerate(model.layers):
        in_features = layer.in_features
        out_features = layer.out_features

        # Архитектура автоэнкодера для текущего слоя
        encoder = nn.Sequential(nn.Linear(in_features, out_features), nn.ReLU())
        decoder = nn.Linear(out_features, in_features)
        ae = nn.Sequential(encoder, decoder).to(device)

        optimizer = optim.Adam(ae.parameters(), lr=LR_AE)
        criterion = nn.MSELoss()

        dataset = TensorDataset(H)
        loader = DataLoader(dataset, batch_size=BATCH, shuffle=True)

        losses = []
        for epoch in range(AE_EPOCHS):
            epoch_loss = 0.0
            for (batch,) in loader:
                batch = batch.to(device)
                optimizer.zero_grad()
                recon = ae(batch)
                loss = criterion(recon, batch)
                loss.backward()
                optimizer.step()
                epoch_loss += loss.item() * len(batch)
            losses.append(epoch_loss / len(H))

        ae_losses.append(losses)
```

```
            # Копируем веса обученного энкодера в основной слой
            layer.weight.data.copy_(encoder[0].weight.data)
            layer.bias.data.copy_(encoder[0].bias.data)

            # Пропускаем данные через обученный слой для следующего этапа
            with torch.no_grad():
                H = encoder(H)

    return ae_losses


# --- 5. Обучение классификатора/регрессора ---
def train_model(model, X_tr, y_tr, X_te, y_te, seed):
    torch.manual_seed(seed)
    model = model.to(device)
    optimizer = optim.Adam(model.parameters(), lr=LR_FT)
    criterion = nn.MSELoss()

    dataset = TensorDataset(X_tr, y_tr)
    loader = DataLoader(dataset, batch_size=BATCH, shuffle=True)

    history = {"train_loss": [], "test_loss": []}

    for epoch in range(FT_EPOCHS):
        model.train()
        epoch_loss = 0.0
        for xb, yb in loader:
            xb, yb = xb.to(device), yb.to(device)
            optimizer.zero_grad()
            pred = model(xb)
            loss = criterion(pred, yb)
            loss.backward()
            optimizer.step()
            epoch_loss += loss.item() * len(xb)
        history["train_loss"].append(epoch_loss / len(X_tr))

        model.eval()
        with torch.no_grad():
            pred_te = model(X_te.to(device))
            test_loss = criterion(pred_te, y_te.to(device)).item()
            history["test_loss"].append(test_loss)

    # Финальные предсказания
    model.eval()
    with torch.no_grad():
        pred = model(X_te.to(device)).cpu().numpy()

    return pred, history


# --- 6. Основной цикл эксперимента ---
def run_experiment():
    X, y = load_parkinsons()

    rows, last = [], {}

    for seed in SEEDS:
        print(f"\n--- Seed {seed} ---")
        X_tr, y_tr, X_te, y_te = prepare(X, y, seed)

        for mode in ["Без предобучения", "С предобучением"]:
            torch.manual_seed(seed)
            model = MLP(X_tr.shape[1], HIDDEN)
            ae_losses = None

            if mode == "С предобучением":
                print("  Предобучение слоев...")
                ae_losses = pretrain_layers(model, X_tr, seed)

            print(f"  Обучение ({mode})...")
            pred, hist = train_model(model, X_tr, y_tr, X_te, y_te, seed)

            # Метрики регрессии
            mae = mean_absolute_error(y_te.numpy(), pred)
            rmse = np.sqrt(mean_squared_error(y_te.numpy(), pred))
            r2 = r2_score(y_te.numpy(), pred)
```

```
                rows.append({
                    "seed": seed, "mode": mode,
                    "MAE": mae, "RMSE": rmse, "R2": r2
                })

                last[mode] = dict(y_te=y_te.numpy(), pred=pred, hist=hist, ae=ae_losses)

        # --- Вывод результатов ---
        res = pd.DataFrame(rows)
        summary = res.groupby("mode")[["MAE", "RMSE", "R2"]].agg(["mean", "std"])
        print(f"\n{'=' * 20} Результаты {'=' * 20}")
        print(summary.round(4).to_string())

        res.to_csv("results_parkinsons.csv", index=False)

        # --- Графики ---
        fig, ax = plt.subplots(1, 2, figsize=(12, 5))
        for i, mode in enumerate(["Без предобучения", "С предобучением"]):
            d = last[mode]
            ax[i].scatter(d["y_te"], d["pred"], alpha=0.5)
            ax[i].plot([d["y_te"].min(), d["y_te"].max()], [d["y_te"].min(), d["y_te"].max()], 'r--')
            ax[i].set_title(f"Предсказания ({mode})")
            ax[i].set_xlabel("Истинные значения")
            ax[i].set_ylabel("Предсказанные значения")
            ax[i].grid(True)
        plt.tight_layout()
        plt.savefig("parkinsons_predictions.png", dpi=150)
        plt.show()

        fig, ax = plt.subplots(1, 2, figsize=(12, 5))
        for i, mode in enumerate(["Без предобучения", "С предобучением"]):
            d = last[mode]
            ax[i].plot(d["hist"]["train_loss"], label="train")
            ax[i].plot(d["hist"]["test_loss"], label="test")
            ax[i].set_title(f"Loss ({mode})")
            ax[i].set_xlabel("Эпоха")
            ax[i].set_ylabel("MSE")
            ax[i].legend()
            ax[i].grid(True)
        plt.tight_layout()
        plt.savefig("parkinsons_loss.png", dpi=150)
        plt.show()

        # График предобучения
        plt.figure(figsize=(8, 5))
        for k, l in enumerate(last["С предобучением"]["ae"]):
            plt.plot(l, label=f"Автоэнкодер слоя {k + 1}")
        plt.xlabel("Эпоха")
        plt.ylabel("MSE")
        plt.title("Предобучение слоев (Parkinsons)")
        plt.legend()
        plt.grid(True)
        plt.tight_layout()
        plt.savefig("parkinsons_ae.png", dpi=150)
        plt.show()


    if __name__ == "__main__":
        run_experiment()
```

Вывод программы:

```
Используемое устройство: cpu
Загрузка данных Parkinsons Telemonitoring...

--- Seed 0 ---
  Обучение (Без предобучения)...
  Предобучение слоев...
  Обучение (С предобучением)...

--- Seed 1 ---
  Обучение (Без предобучения)...
  Предобучение слоев...
  Обучение (С предобучением)...

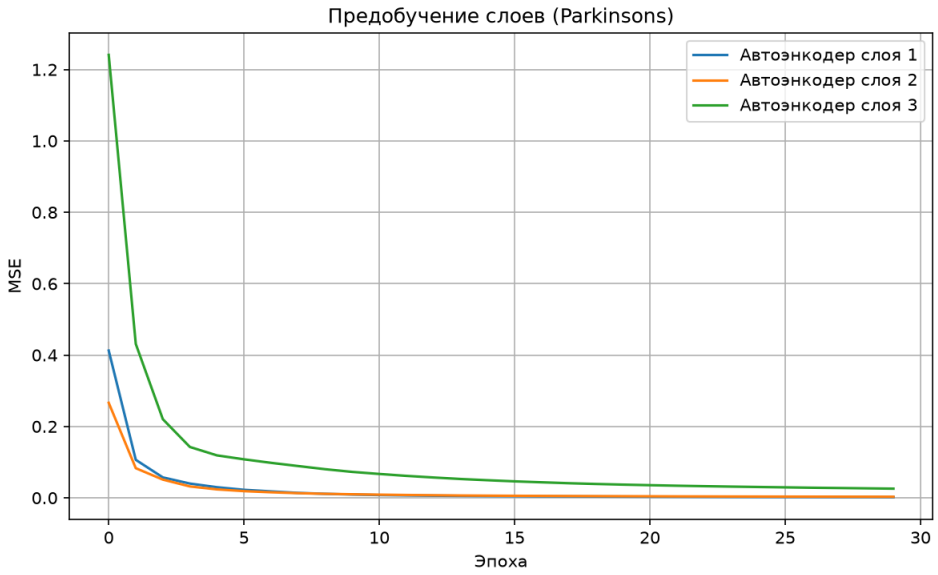
--- Seed 2 ---
  Обучение (Без предобучения)...
  Предобучение слоев...
  Обучение (С предобучением)...

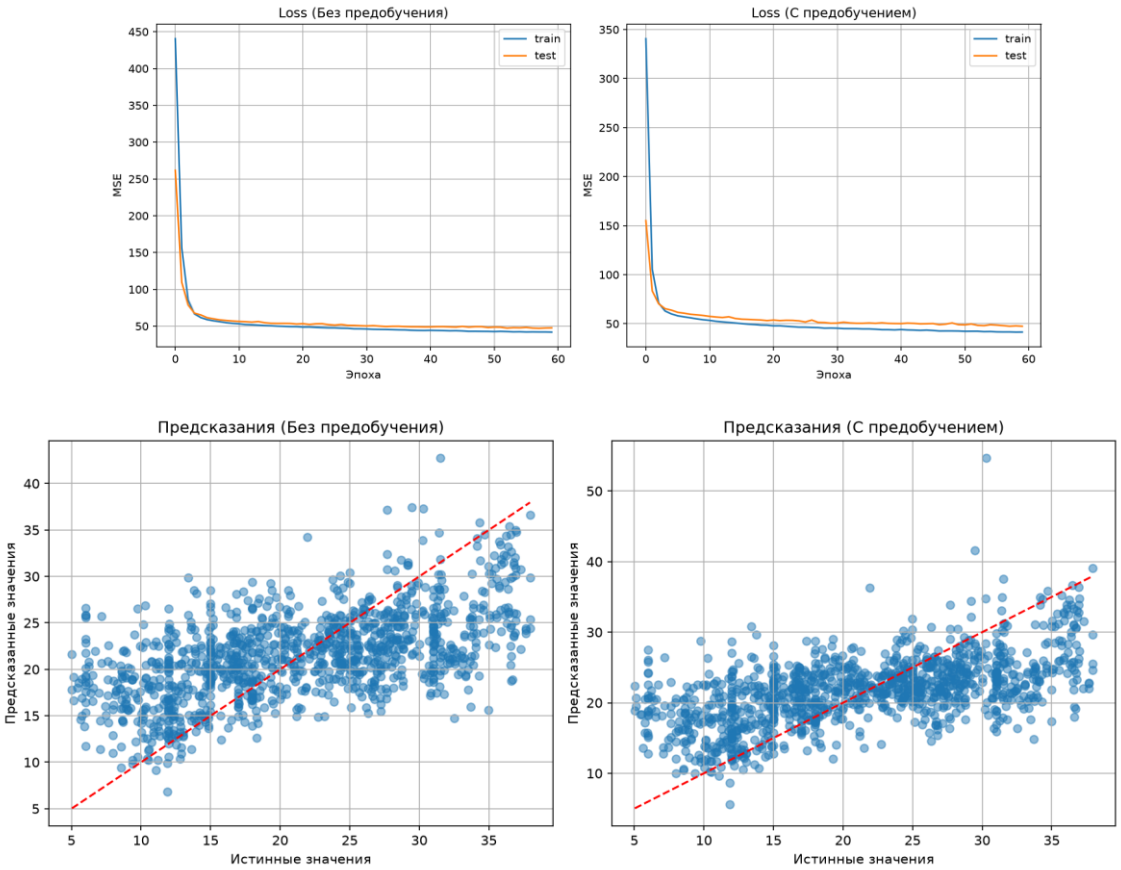
--- Seed 3 ---
  Обучение (Без предобучения)...
  Предобучение слоев...
  Обучение (С предобучением)...

--- Seed 4 ---
  Обучение (Без предобучения)...
  Предобучение слоев...
  Обучение (С предобучением)...

==================== Результаты ====================
                    MAE            RMSE              R2
                   mean     std    mean     std    mean     std
mode
Без предобучения  5.5861  0.1386  6.9109  0.1219  0.2838  0.0423
С предобучением   5.5431  0.1127  6.9082  0.1136  0.2843  0.0434
```

Графики:





В работе сравнивалось обучение полносвязной нейронной сети из четырех слоев (архитектура 64→32→16→1) без предобучения и с послойным автоэнкодерным предобучением на пяти случайных разбиениях (SEEDS). Задача относилась к типу **регрессии**, поэтому качество оценивалось с помощью метрик **MAE** (средняя абсолютная ошибка), **RMSE** (среднеквадратичная ошибка) и **R^2** (коэффициент детерминации).

На датасете *Parkinsons Telemonitoring* без предобучения были получены следующие результаты: MAE = **5.5861 ± 0.1386**, RMSE = **6.9109 ± 0.1219**, R^2 = **0.2838 ± 0.0423**. При использовании автоэнкодерного предобучения метрики составили: MAE = **5.5431 ± 0.1127**, RMSE = **6.9082 ± 0.1136**, R^2 = **0.2843 ± 0.0434**.

Сравнение показывает, что предобучение дало очень незначительный прирост качества: MAE улучшилось всего на 0.043, а R^2 — на 0.0005, что находится в пределах стандартного отклонения между запусками и не является статистически значимым. Это объясняется тем, что архитектура сети не является глубокой, а объем данных достаточен для обучения с нуля. Кроме того, предсказание степени болезни Паркинсона (motor_UPDRS) — сложная задача, где нелинейные зависимости не всегда улавливаются простым автоэнкодером.

Таким образом, послойное предобучение не улучшило итоговое качество модели на данном датасете, однако оно может быть полезным при работе с очень глубокими сетями или при дефиците размеченных данных в качестве метода регуляризации.

Вывод: научился осуществлять предобучение нейронных сетей с помощью автоэнкодерного подхода и сравнивать эффективность моделей с предобучением и без него.